CO 4 - ASSESSMENT TOOL - 1

Name : Lanka Kavya
Registration Number : 192472298
Subject Code : CSA0619
Subject Name : DAA

Q17 : Subset Sum Problem

Given a set of integers and a target value, determine whether there exists a subset whose sum is exactly equal to the target. Each element can either be included once or excluded from the subset. The objective is to check the existence of at least one valid subset combination. Use Dynamic Programming to solve the problem efficiently.
Input Format:
n = number of elements
arr[] = set elements
sum = target sum

Sol :

Source Code :

```
n = int(input("Enter number of elements: "))

arr = list(map(int, input("Enter elements: ").split()))

target = int(input("Enter target sum: "))

dp = [False] * (target + 1)

dp[0] = True

for x in arr:

    for j in range(target, x - 1, -1):

        dp[j] = dp[j] or dp[j - x]

if dp[target]:

    print("Dynamic Programming: Subset Exists = Yes")

else:

    print("Dynamic Programming: Subset Exists = No")

arr.sort(reverse=True)
```

```
remaining = target

for x in arr:

    if x <= remaining:

        remaining -= x

if remaining == 0:

    print("Greedy Strategy: Subset Exists = Yes")

else:

    print("Greedy Strategy: Subset Exists = No")
```

Subset Sum Problem – Report

The Subset Sum Problem is a problem in which we are given a set of integers and a target value. The objective is to determine whether there is any subset of the given elements whose sum is exactly equal to the target value. Each element can be selected at most once.

For the given problem, the user enters the number of elements, the array elements, and the target sum. The program solves the problem using both Dynamic Programming and Greedy Strategy.

Dynamic Programming

Dynamic Programming stores the results of previously calculated sub problems to avoid repeated calculations. A DP array is created where each position represents whether a particular sum can be obtained using the elements processed so far.

For every element, the program checks whether the current sum can be obtained either by excluding the element or by including it. The DP array is updated from right to left so that an element is not used more than once.

For example:

arr = [3, 34, 4, 12, 5]

target = 9

The subset: 4 + 5 = 9

exists, so the output is:

Subset Exists = Yes

Greedy Strategy

The Greedy approach sorts the elements in descending order and selects the largest element that does not exceed the remaining target. The selected element is subtracted from the target, and the process continues.

For the given example, the elements are sorted as: 34, 12, 5, 4, 3

The algorithm selects `5`, leaving `4`, and then selects `4`.

Therefore: 5 + 4 = 9

So the Greedy approach also gives:

Subset Exists = Yes

Comparison

Dynamic Programming is more reliable because it considers all possible combinations required to determine whether the target sum can be formed. Its time complexity is O(n × target). The Greedy approach is faster in general after sorting, with a time complexity of O(n log n), but it does not always produce the correct answer for the Subset Sum Problem because choosing the largest available element can prevent a valid combination from being found.

Therefore, Dynamic Programming is the preferred approach for obtaining a guaranteed correct solution, while Greedy can work for some specific inputs.

Output Screenshots :